

LU Factorization

Stefan Kiefer

Department of Computer Science
University of Oxford

This starts the part of the course that is not mandatory for CS&Phil students.

The material from these slides is adapted from

[Trefethen, Bau. *Numerical Linear Algebra*],

not from the Hefferon textbook used in the course up to now.

LU Factorization via Gaussian Elimination

$$\begin{pmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \end{pmatrix} \xrightarrow{L_1} \begin{pmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \end{pmatrix} \xrightarrow{L_2} \begin{pmatrix} \times & \times & \times & \times \\ & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \end{pmatrix} \xrightarrow{L_3} \begin{pmatrix} \times & \times & \times & \times \\ & \times & \times & \times \\ & & \times & \times \\ & & & 0 & \times \end{pmatrix}$$

A
 $L_1 A$
 $L_2 L_1 A$
 $L_3 L_2 L_1 A$

The L_i are lower-triangular matrices, multiplied from the left. They are products of elementary reduction matrices $C_{i,j}(k)$. Recall that $C_{i,j}(k)$ corresponds to the Gaussian operation “ $k\rho_i + \rho_j$ ”. So, in fact, the L_i have 1s on the diagonal: they are *unit lower-triangular*.

We are assuming that A is an $n \times n$ square matrix and no row exchanges are necessary.

$$\underbrace{L_{n-1} \cdots L_2 L_1}_{L^{-1}} A = U$$

Setting $L := L_1^{-1} L_2^{-1} \cdots L_{n-1}^{-1}$ (also a lower-triangular matrix) gives an *LU factorization*

$$A = LU,$$

i.e., L is lower-triangular and U is upper-triangular.

Example

$$A = \begin{pmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{pmatrix}$$

Three steps of Gauss's Method (in one) correspond to multiplication with a matrix L_1 :

$$L_1 A = \begin{pmatrix} 1 & & & \\ -2 & 1 & & \\ -4 & & 1 & \\ -3 & & & 1 \end{pmatrix} \begin{pmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{pmatrix} = \begin{pmatrix} 2 & 1 & 1 & 0 \\ & 1 & 1 & 1 \\ & 3 & 5 & 5 \\ & 4 & 6 & 8 \end{pmatrix}$$

Considering the second column:

$$L_2 L_1 A = \begin{pmatrix} 1 & & & \\ & 1 & & \\ -3 & & 1 & \\ -4 & & & 1 \end{pmatrix} \begin{pmatrix} 2 & 1 & 1 & 0 \\ & 1 & 1 & 1 \\ & 3 & 5 & 5 \\ & 4 & 6 & 8 \end{pmatrix} = \begin{pmatrix} 2 & 1 & 1 & 0 \\ & 1 & 1 & 1 \\ & & 2 & 2 \\ & & 2 & 4 \end{pmatrix}$$

Finally, we subtract the third row from the fourth:

$$L_3 L_2 L_1 A = \begin{pmatrix} 1 & & & \\ & 1 & & \\ & & 1 & \\ & & -1 & 1 \end{pmatrix} \begin{pmatrix} 2 & 1 & 1 & 0 \\ & 1 & 1 & 1 \\ & & 2 & 2 \\ & & 2 & 4 \end{pmatrix} = \begin{pmatrix} 2 & 1 & 1 & 0 \\ & 1 & 1 & 1 \\ & & 2 & 2 \\ & & & 2 \end{pmatrix} =: U$$

For the factorization $A = LU$ we need $L = L_1^{-1} L_2^{-1} L_3^{-1}$. This turns out to be easy (“two strokes of luck”). Firstly,

$$\begin{pmatrix} 1 & & & \\ -2 & 1 & & \\ -4 & & 1 & \\ -3 & & & 1 \end{pmatrix}^{-1} = \begin{pmatrix} 1 & & & \\ 2 & 1 & & \\ 4 & & 1 & \\ 3 & & & 1 \end{pmatrix} \text{ etc.}$$

Secondly, the product $L_1^{-1} L_2^{-1} L_3^{-1}$ is obtained by inserting the nonzero subdiagonal elements at the “natural” places. Altogether:

$$\begin{matrix} \begin{pmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{pmatrix} \\ A \end{matrix} = \begin{matrix} \begin{pmatrix} 1 & & & \\ 2 & 1 & & \\ 4 & 3 & 1 & \\ 3 & 4 & 1 & 1 \end{pmatrix} \\ L \end{matrix} \begin{matrix} \begin{pmatrix} 2 & 1 & 1 & 0 \\ & 1 & 1 & 1 \\ & & 2 & 2 \\ & & & 2 \end{pmatrix} \\ U \end{matrix}$$

General formula

Suppose $\vec{x}_k$ denotes the k th column of the matrix before (macro-)step k . Then L_k should be chosen so that

$$\vec{x}_k = \begin{pmatrix} x_{1k} \\ \vdots \\ x_{kk} \\ x_{k+1,k} \\ \vdots \\ x_{nk} \end{pmatrix} \xrightarrow{L_k} L_k \vec{x}_k = \begin{pmatrix} x_{1k} \\ \vdots \\ x_{kk} \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

To do this, we subtract ℓ_{jk} times row k from row $j > k$, where ℓ_{jk} is the “multiplier” $\ell_{jk} := \frac{x_{jk}}{x_{kk}}$. This gives:

$$L_k = \begin{pmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & & & \\ & & -\ell_{k+1,k} & 1 & & \\ & & \vdots & & \ddots & \\ & & -\ell_{nk} & & & 1 \end{pmatrix}$$

First stroke of luck

$= L_k$ can be inverted by negating its subdiagonal entries

Define:

$$\vec{\ell}_k := \begin{pmatrix} 0 \\ \vdots \\ 0 \\ \ell_{k+1,k} \\ \vdots \\ \ell_{n,k} \end{pmatrix}$$

Then L_k can be written

$$L_k = I - \vec{\ell}_k \vec{e}_k^\top.$$

By the sparsity pattern of $\vec{\ell}_k$ we have $\vec{e}_k^\top \vec{\ell}_k = 0$. Therefore

$$(I - \vec{\ell}_k \vec{e}_k^\top)(I + \vec{\ell}_k \vec{e}_k^\top) = I - \vec{\ell}_k \vec{e}_k^\top \vec{\ell}_k \vec{e}_k^\top = I$$

and so $L_k^{-1} = (I + \vec{\ell}_k \vec{e}_k^\top)$ as claimed.

Second stroke of luck

$= L_1^{-1} L_2^{-1} \cdots L_{n-1}^{-1}$ has the multipliers at their “natural” places

Consider $L_k^{-1} L_{k+1}^{-1}$. From the sparsity pattern of $\vec{\ell}_{k+1}$ we have $\vec{e}_k^\top \vec{\ell}_{k+1} = 0$ and therefore:

$$L_k^{-1} L_{k+1}^{-1} = (I + \vec{\ell}_k \vec{e}_k^\top)(I + \vec{\ell}_{k+1} \vec{e}_{k+1}^\top) = I + \vec{\ell}_k \vec{e}_k^\top + \vec{\ell}_{k+1} \vec{e}_{k+1}^\top$$

This is a unit lower-triangular matrix with the multipliers at their natural places below the diagonal. When we take the product of all of these matrices to form L , we have the same convenient property everywhere below the diagonal:

$$L = L_1^{-1} L_2^{-1} \cdots L_{n-1}^{-1} = \begin{pmatrix} 1 & & & & \\ \ell_{21} & 1 & & & \\ \ell_{31} & \ell_{32} & 1 & & \\ \vdots & \vdots & \ddots & \ddots & \\ \ell_{n1} & \ell_{n2} & \cdots & \ell_{n,n-1} & 1 \end{pmatrix}$$

In practice, one stores only the multipliers ℓ_{jk} , not the matrices L_k or L_k^{-1} .

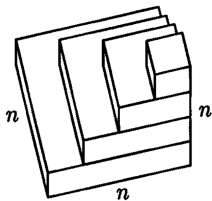
Algorithm 1: LU Factorization without Pivoting

```
U := A; L := I
for k := 1 to n - 1
    for j := k + 1 to n
        ljk :=  $\frac{u_{jk}}{u_{kk}}$ 
        uj,k:n := uj,k:n - ljkuk,k:n
```

The notation $u_{j,k:n}$ means a row with entries $u_{jk}, u_{j,k+1}, \dots, u_{jn}$ (inspired by Matlab, often used in linear algebra).

To minimize memory use, one might use only one array, containing both L and U, leaving the 1s on the diagonal of L implicit.

Two floating-point operations per modified entry (one multiplication, one subtraction). Overall 2 times the volume of this “pyramid”



which is approximately $2 \cdot \frac{1}{3}n^3$ in the limit.

Solving Linear Systems via LU Factorization

Suppose we have a linear system $A\vec{x} = \vec{b}$ and an LU factorization $A = LU$. Then:

$$L \underbrace{U\vec{x}}_{\vec{y}} = \vec{b}$$

So we can solve the system by first solving $L\vec{y} = \vec{b}$ and then $U\vec{x} = \vec{y}$.

Thanks to the sparsity pattern of L , the system $L\vec{y} = \vec{b}$ can be efficiently solved by “forward substitution” (in the natural way, top down). Similarly, then solve $U\vec{x} = \vec{y}$ by backward substitution (bottom up). Each of those requires about n^2 flops, much less than needed for computing the LU factorization. The overall cost (including computing the LU factorization) is about $\frac{2}{3}n^3$.

This works well also for multiple linear systems with the same matrix A and different right-hand sides $\vec{b}$:

Don't invert A . Instead (LU-)factorize A !

Pivoting and PLU Factorization

Instability without Pivoting

We know from the beginning of the course that in general we need row swaps; e.g., here:

$$A = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$$

Strictly speaking, we do not need a row swap here:

$$A = \begin{pmatrix} 10^{-20} & 1 \\ 1 & 1 \end{pmatrix}$$

The LU factorization algorithm as described subtracts 10^{20} times the first row from the second row, producing the factors:

$$L = \begin{pmatrix} 1 & 0 \\ 10^{20} & 1 \end{pmatrix} \quad U = \begin{pmatrix} 10^{-20} & 1 \\ 0 & 1 - 10^{20} \end{pmatrix}$$

A floating-point representation might store only, say, 16 leading digits. Rounding the bottom-right entry to the nearest such number gives

$$\tilde{L} = \begin{pmatrix} 1 & 0 \\ 10^{20} & 1 \end{pmatrix} \quad \tilde{U} = \begin{pmatrix} 10^{-20} & 1 \\ 0 & -10^{20} \end{pmatrix}$$

Multiplying $\tilde{\tilde{L}}\tilde{\tilde{U}}$ gives

$$\tilde{\tilde{L}}\tilde{\tilde{U}} = \begin{pmatrix} 10^{-20} & 1 \\ 1 & 0 \end{pmatrix} \quad \text{vs.} \quad A = \begin{pmatrix} 10^{-20} & 1 \\ 1 & 1 \end{pmatrix}$$

Is that bad?

Solving

$$\begin{pmatrix} 10^{-20} & 1 \\ 1 & 0 \end{pmatrix} \vec{x} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \text{gives} \quad \vec{x} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

whereas the correct solution of

$$\begin{pmatrix} 10^{-20} & 1 \\ 1 & 1 \end{pmatrix} \vec{x} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \text{is} \quad \vec{x} \approx \begin{pmatrix} -1 \\ 1 \end{pmatrix}.$$

Yes, the error is bad.

We conclude that row swaps are needed for stability reasons, even when the LU factorization algorithm as given does not divide by 0.

Row swaps avoid stability issues, but that is not easy to show.

Pivots

We have seen:

$$\begin{pmatrix} \times & \times & \times & \times & \times \\ & x_{kk} & \times & \times & \times \\ & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & \times & \times & \times & \times \end{pmatrix} \longrightarrow \begin{pmatrix} \times & \times & \times & \times & \times \\ & x_{kk} & \times & \times & \times \\ & 0 & \times & \times & \times \\ & 0 & \times & \times & \times \\ & 0 & \times & \times & \times \end{pmatrix}$$

Here, x_{kk} is called *pivot*. One might choose another pivot, either because x_{kk} is zero or small. This is best handled with a row swap:

$$\begin{pmatrix} \times & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & x_{ik} & \times & \times & \times \\ & \times & \times & \times & \times \end{pmatrix} \xrightarrow{P_1} \begin{pmatrix} \times & \times & \times & \times & \times \\ & x_{ik} & \times & \times & \times \\ & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & \times & \times & \times & \times \end{pmatrix} \xrightarrow{L_1} \begin{pmatrix} \times & \times & \times & \times & \times \\ & x_{ik} & \times & \times & \times \\ & 0 & \times & \times & \times \\ & 0 & \times & \times & \times \\ & 0 & \times & \times & \times \end{pmatrix}$$

Pivot selection Row swap Elimination

$n - 1$ steps of this gives $L_{n-1}P_{n-1} \cdots L_2P_2L_1P_1A = U$ upper-triangular.

Example

$$A = \begin{pmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{pmatrix}$$

Swap rows 1 and 3:

$$P_1 A = \begin{pmatrix} & & 1 & \\ & 1 & & \\ 1 & & & \\ & & & 1 \end{pmatrix} \begin{pmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{pmatrix} = \begin{pmatrix} 8 & 7 & 9 & 5 \\ 4 & 3 & 3 & 1 \\ 2 & 1 & 1 & 0 \\ 6 & 7 & 9 & 8 \end{pmatrix}$$

Elimination (produce 0s in the first column):

$$L_1 P_1 A = \begin{pmatrix} 1 & & & \\ -\frac{1}{2} & 1 & & \\ -\frac{1}{4} & & 1 & \\ -\frac{3}{4} & & & 1 \end{pmatrix} \begin{pmatrix} 8 & 7 & 9 & 5 \\ 4 & 3 & 3 & 1 \\ 2 & 1 & 1 & 0 \\ 6 & 7 & 9 & 8 \end{pmatrix} = \begin{pmatrix} 8 & 7 & 9 & 5 \\ -\frac{1}{2} & -\frac{3}{2} & -\frac{3}{2} & -\frac{3}{2} \\ -\frac{3}{4} & -\frac{5}{4} & -\frac{5}{4} & -\frac{5}{4} \\ \frac{7}{4} & \frac{9}{4} & \frac{17}{4} & \frac{17}{4} \end{pmatrix}$$

Swap rows 2 and 4:

$$\begin{pmatrix} 1 & & & \\ & & & 1 \\ & 1 & & \\ & & 1 & \end{pmatrix} \begin{pmatrix} 8 & 7 & 9 & 5 \\ & -\frac{1}{2} & -\frac{3}{2} & -\frac{3}{2} \\ -\frac{3}{4} & -\frac{5}{4} & -\frac{5}{4} & \\ \frac{7}{4} & \frac{9}{4} & \frac{17}{4} & \end{pmatrix} = \begin{pmatrix} 8 & 7 & 9 & 5 \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ -\frac{3}{4} & -\frac{5}{4} & -\frac{5}{4} & \\ -\frac{1}{2} & -\frac{3}{2} & -\frac{3}{2} & \end{pmatrix}$$

Elimination (produce 0s in the second column):

$$\begin{pmatrix} 1 & & & \\ & 1 & & \\ & \frac{3}{7} & 1 & \\ & \frac{2}{7} & & 1 \end{pmatrix} \begin{pmatrix} 8 & 7 & 9 & 5 \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ -\frac{3}{4} & -\frac{5}{4} & -\frac{5}{4} & \\ -\frac{1}{2} & -\frac{3}{2} & -\frac{3}{2} & \end{pmatrix} = \begin{pmatrix} 8 & 7 & 9 & 5 \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ & & -\frac{2}{7} & \frac{4}{7} \\ & & -\frac{6}{7} & -\frac{2}{7} \end{pmatrix}$$

Swap rows 3 and 4:

$$\begin{pmatrix} 1 & & & \\ & 1 & & \\ & & & 1 \\ & & 1 & \end{pmatrix} \begin{pmatrix} 8 & 7 & 9 & 5 \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ -\frac{2}{7} & \frac{4}{7} & & \\ -\frac{6}{7} & -\frac{2}{7} & & \end{pmatrix} = \begin{pmatrix} 8 & 7 & 9 & 5 \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ & & -\frac{6}{7} & -\frac{2}{7} \\ & & -\frac{2}{7} & \frac{4}{7} \end{pmatrix}$$

Elimination (produce a 0 in the third column):

$$\begin{pmatrix} 1 & & & \\ & 1 & & \\ & & 1 & \\ & & -\frac{1}{3} & 1 \end{pmatrix} \begin{pmatrix} 8 & 7 & 9 & 5 \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ & & -\frac{6}{7} & -\frac{2}{7} \\ & & -\frac{2}{7} & \frac{4}{7} \end{pmatrix} = \begin{pmatrix} 8 & 7 & 9 & 5 \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ & & -\frac{6}{7} & -\frac{2}{7} \\ & & & \frac{2}{3} \end{pmatrix}$$

Put altogether we got:

$$\underset{\text{A}}{\text{L}_3 \text{P}_3 \text{L}_2 \text{P}_2 \text{L}_1 \text{P}_1} \begin{pmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{pmatrix} = \begin{pmatrix} 8 & 7 & 9 & 5 \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ & & -\frac{6}{7} & -\frac{2}{7} \\ & & & \frac{2}{3} \end{pmatrix} \underset{\text{U}}{\quad}$$

This actually works out like this (not obvious why):

$$\underset{\text{P}}{\begin{pmatrix} & & 1 & \\ & & & 1 \\ & 1 & & \\ 1 & & & \end{pmatrix}} \underset{\text{A}}{\begin{pmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{pmatrix}} = \underset{\text{L}}{\begin{pmatrix} 1 & & & \\ \frac{3}{4} & 1 & & \\ \frac{1}{2} & -\frac{2}{7} & 1 & \\ \frac{1}{4} & -\frac{3}{7} & \frac{1}{3} & 1 \end{pmatrix}} \underset{\text{U}}{\begin{pmatrix} 8 & 7 & 9 & 5 \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ & & -\frac{6}{7} & -\frac{2}{7} \\ & & & \frac{2}{3} \end{pmatrix}}$$

The subdiagonal elements of L are ≤ 1 in magnitude.

Third stroke of luck = The P s and L s can be separated

We had:

$$L_3 P_3 L_2 P_2 L_1 P_1 A = U$$

The row operations can be reordered:

$$L_3 P_3 L_2 P_2 L_1 P_1 = L'_3 L'_2 L'_1 P_3 P_2 P_1,$$

where L'_k is equal to L_k but with the subdiagonal entries permuted. More precisely, define:

$$L'_3 := L_3, \quad L'_2 := P_3 L_2 P_3^{-1}, \quad L'_1 := P_3 P_2 L_1 P_2^{-1} P_3^{-1}$$

Here, each L_k “suffers” only permutations P_j for $j > k$, so L'_k has the same structure as L_k . And we have

$$L'_3 L'_2 L'_1 P_3 P_2 P_1 = L_3 (P_3 L_2 P_3^{-1}) (P_3 P_2 L_1 P_2^{-1} P_3^{-1}) P_3 P_2 P_1 = L_3 P_3 L_2 P_2 L_1 P_1$$

as claimed.

In this way we get in general

$$(L'_{n-1} \cdots L'_2 L'_1)(P_{n-1} \cdots P_2 P_1)A = U$$

The product of the L'_k is again unit lower-triangular and can be inverted as before by negating the subdiagonal entries. Defining

$$L := (L'_{n-1} \cdots L'_2 L'_1)^{-1} \quad \text{and} \quad P := P_{n-1} \cdots P_2 P_1$$

we get

$$PA = LU,$$

a *PLU factorization* of A . The row swaps (= *partial pivoting*) are common-place, so PLU factorizations are often just called LU factorizations. (*Complete* pivoting would also involve column swaps; not often considered.)

Gaussian elimination with partial pivoting therefore has the following interpretation:

- 1) Permute the rows of A according to P .
- 2) Apply Gaussian elimination without pivoting to PA .

This is not an algorithm, as P is not known beforehand!

Algorithm 2: LU Factorization with Partial Pivoting

```
U := A; L := I; P := I
for k := 1 to n - 1
    Select i ≥ k to maximize |uik|
    uk,k:n ↔ ui,k:n (swap rows)
    ℓk,1:k-1 ↔ ℓi,1:k-1
    pk,: ↔ pi,:
    for j := k + 1 to n
        ℓjk :=  $\frac{u_{jk}}{u_{kk}}$ 
        uj,k:n := uj,k:n - ℓjkuk,k:n
```

The asymptotic number of flops is still $\frac{2}{3}n^3$, as before without pivoting. In practice, the permutation is not stored as a matrix P, but an array of n indices. Again, L and U might be stored in the same array.